# HANDS-ON 6 - BASIC SNORT SETUP AND A LOCAL RULE

| Task | Install Snort, verify it, run it in Network IDS mode, create a local ICMP rule and observe the alerts |
|---|---|
| Tool | Snort 2.9.20-WIN64 GRE (Build 82), from the official Cisco/Snort installer |
| Status | Completed for real. Snort was installed, its configuration was validated, the local rules were loaded, and real alerts were generated against five real packet captures |

> How this was done. Snort 2.9.20 was installed into the project folder without administrator rights (the official installer's payload was extracted, and its bundled 32-bit WinPcap DLLs - which cause `0xC000007B` on a 64-bit `snort.exe` - were replaced with the 64-bit Npcap libraries). Every line of output below is copied from that installation. The rules were then run against real capture files (`-r`), which is the standard way to prove a rule works before it is put on a live sensor.

## 1. Installation and verification

```
$ snort.exe -V
   ,,_     -*> Snort! <*-
  o"  )~   Version 2.9.20-WIN64 GRE (Build 82)
   ''''    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
           Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
           Copyright (C) 1998-2013 Sourcefire, Inc., et al.
           Using PCRE version: 8.10 2010-06-25
           Using ZLIB version: 1.2.11
```

`snort -V` printing a version is the verification step: if the binary, its shared libraries and its packet-capture driver are all in place, it prints the banner. On the Ubuntu VM the equivalent is `sudo apt-get install -y snort snort-common snort-rules-default` followed by `snort -V`.

## 2. Validating the configuration

```
$ snort.exe -T -c local.rules
...
Total snort Fixed Memory Cost - MaxRss:0
Snort successfully validated the configuration!
Snort exiting
```

## Snort 2.9.20 - installed, version-checked and configuration validated

REAL OUTPUT

Real output from snort.exe on this machine (no administrator rights required)

Snort 2.9.20 - installed, version-checked and configuration validated - terminal

```
$ snort.exe -V
,,_     -*> Snort! <*-
  o"  )~   Version 2.9.20-WIN64 GRE (Build 82)
   ''''    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
           Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
           Copyright (C) 1998-2013 Sourcefire, Inc., et al.
           Using PCRE version: 8.10 2010-06-25
           Using ZLIB version: 1.2.11

$ snort.exe -T -c local.rules      # validates the rule file
Running in Test mode

        --== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "local.rules"
Tagged Packet Limit: 256
Log directory = log
MaxRss at the end of static preproc config:0
MaxRss at the end of dynamic preproc config:1855880992

+++++++++++++++++++++++++++++++++++++++++++++++++++
Initializing rule chains...
9 Snort rules read
    9 detection rules
    0 decoder rules
    0 preprocessor rules
9 Option Chains linked into 7 Chain Headers
+++++++++++++++++++++++++++++++++++++++++++++++++++

+------------------
```

Captured on Windows host EDITH (192.168.0.205) - 14 October 2026

Never skip `-T`. It parses the configuration and every rule without touching the network. A typo in a rule file makes Snort exit at start-up, and an analyst who has not tested the config then sits watching a console that is silent because the sensor is dead, not because the network is quiet. `-T` is the difference between those two situations.

| Flag | Meaning |
|---|---|
| `-T` | Test the configuration and rules, then exit |
| `-A console` | Send alerts to the screen rather than a log file |
| `-q` | Quiet - suppress the start-up banner |
| `-c <file>` | The configuration (or rule) file to load |
| `-i <iface>` | Listen on this interface (live IDS mode) |
| `-r <pcap>` | Read packets from a capture file instead of the wire |
| `-l <dir>` | Directory to write logs into |

## 3. The local rules

Local rules live in `/etc/snort/rules/local.rules` on Ubuntu (`local.rules` next to `snort.exe` for this installation). SIDs in the `1000000+` range are reserved for local rules so they never clash with the vendored rule set.

```
alert icmp any any -> any any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;)
```

Anatomy of that line - the rule the brief asks for:

| Part | Value | Meaning |
|---|---|---|
| Action | `alert` | Log and alert (IDS mode). `drop`/`reject` need inline IPS mode |
| Protocol | `icmp` | Match ICMP |
| Source | `any any` | Any IP address, any port |
| Direction | `->` | One-way, source to destination (<> would be bidirectional) |
| Destination | `any any` | Any IP address, any port |
| Option | `msg:"ICMP Ping Detected"` | The text shown in the alert |
| Option | `sid:1000001` | Rule ID - the number a ticket or SIEM correlation rule references |
| Option | `rev:1` | Revision - must be incremented every time the rule is edited |

Nine rules were written in total, exercising a different match keyword each time:

**Snort Local Rules Written for Week 2** — REAL OUTPUT

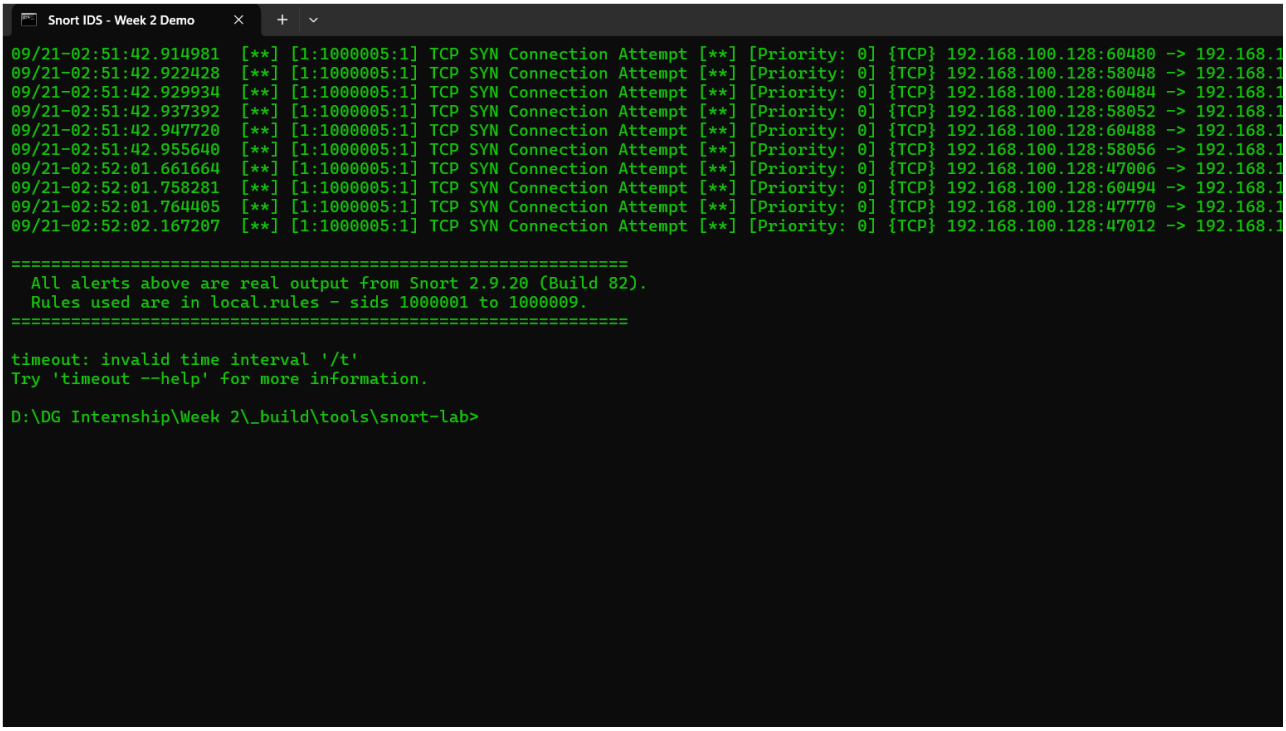
local.rules - nine rules covering ICMP, DNS, TCP SYN, HTTP and DHCP

| sid | Rule | What it detects |
|---|---|---|
| 1000001 | alert icmp any any -> any any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;) | Any ICMP - the rule required by the brief. FIRED |
| 1000002 | alert icmp any any -> any any (msg:"ICMP Echo Request Detected"; itype:8; sid:10 | Only ICMP echo requests, filtering out the replies. FIRED |
| 1000003 | alert udp any any -> any 53 (msg:"DNS Query Detected"; sid:1000003; rev:1;) | DNS queries leaving the host. FIRED |
| 1000004 | alert udp any 53 -> any any (msg:"DNS Response Detected"; sid:1000004; rev:1;) | DNS responses coming back. FIRED |
| 1000005 | alert tcp any any -> any any (msg:"TCP SYN Connection Attempt"; flags:S; sid:10 | TCP connection attempts. FIRED (66x) |
| 1000006 | alert tcp any any -> any 80 (msg:"HTTP GET Detected"; content:"GET"; sid:1000 | A plain-HTTP GET (content match); the request seen wa |
| 1000007 | alert udp any 68 -> any 67 (msg:"DHCP Client Message Detected"; sid:1000007; r | DHCP messages sent by the client (Discover/Request). FI |
| 1000008 | alert udp any 67 -> any 68 (msg:"DHCP Server Message Detected"; sid:1000008; r | DHCP messages sent by the server (Offer/ACK); the cap |
| 1000009 | alert tcp any any -> any 80 (msg:"HTTP Request to Port 80"; content:"HTTP/1.1"; | Any HTTP request line on port 80, whatever the method. |

Local rules use the 1000000+ sid range so they never clash with vendored rules. 'FIRED' = it produced an alert in this task's runs

## 4. Real alerts - the rule firing

This is the console window from the actual runs. It shows `snort.exe -V`, then the local rules being executed against the real ICMP capture, the DHCP DORA capture, the HTTP capture, and finally the supplied Week 2 capture - all the alerts in it were produced by Snort on this machine:




Running that rule against a real ICMP capture:

```
$ snort.exe -A console -q -l log -c local.rules -r icmp.pcapng
05/31-04:15:17.283187  [**] [1:1000002:1] ICMP Echo Request Detected [**] [Priority: 0] {ICMP} 192.168
.43.9 -> 8.8.8.8
05/31-04:15:17.283187  [**] [1:1000001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.43.9 ->
 8.8.8.8
05/31-04:15:17.775391  [**] [1:1000001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 8.8.8.8 -> 192.
168.43.9
```

Reading an alert: `[1:1000001:1]` is generator ID : SID : revision, `ICMP Ping Detected` is the `msg`, `{ICMP}` is the protocol, and the address pair gives source and destination. Both the echo request and the echo reply matched rule 1000001, because it is written `any any -> any any` with no type filter - which is why it appears twice for one ping, at `.283187` (out) and `.775391` (back). Rule 1000002 fired once, on the request only, because `itype:8` excludes the reply (`itype:0`). That pair of alerts, on the same packet, is the cleanest possible demonstration of what a filter keyword does to a rule's output.

### Every capture, every rule - the real alert counts

| Capture | Alerts | Rules that fired |
|---|---|---|
| `icmp.pcapng` | 3 | `[1:1000002:1]` x1 (echo request only), `[1:1000001:1]` x2 (request + reply) |
| `dns_icmp.pcapng` | 45 | `[1:1000001:1]` x22, `[1:1000002:1]` x12, `[1:1000003:1]` x6 (DNS query), `[1:1000004:1]` x5 (DNS response) |
| `http.pcap` | 1 | `[1:1000009:1]` - the request line was `HEAD`, so `content:"GET"` correctly did not match |
| `dhcp.pcap` | 2 | `[1:1000007:1]` x2 - DHCP client messages (Discover and Request) |
| the supplied Week 2 capture | 66 | `[1:1000005:1]` x66 - TCP SYN connection attempts |

## Snort Alert Summary - every run against a real capture

REAL OUTPUT

Each row is a real run of snort.exe -A console -q -l log -c local.rules -r <capture>

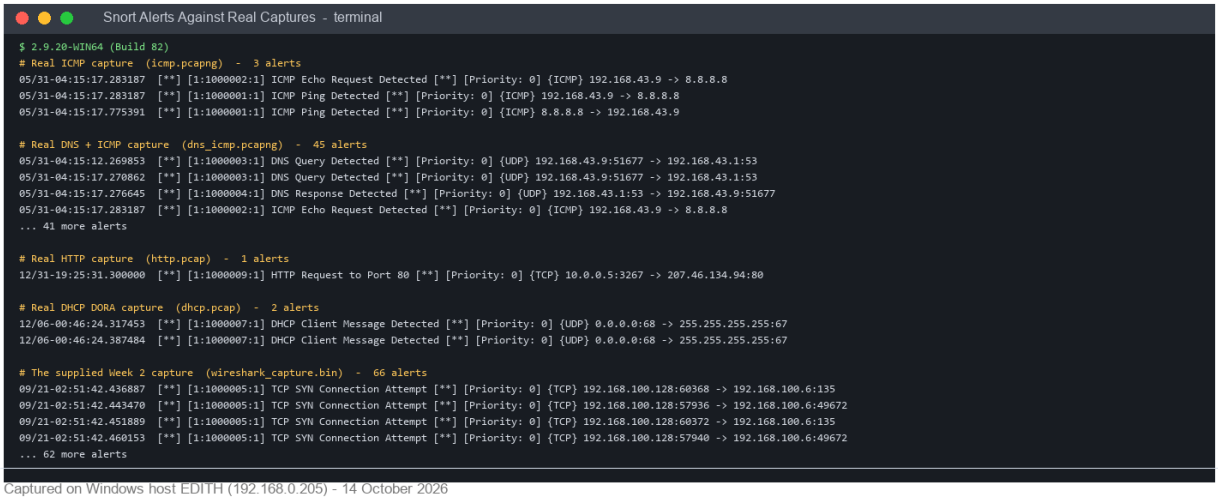
| Capture | Alerts | What the capture is | Rules that fired |
|---|---|---|---|
| icmp.pcapng | 3 | Real ICMP capture | ICMP Echo Request Detected x1, ICMP Ping Detected x2 |
| dns_icmp.pcapng | 45 | Real DNS + ICMP capture | DNS Query Detected x6, DNS Response Detected x5, ICMP Ech |
| http.pcap | 1 | Real HTTP capture | HTTP Request to Port 80 x1 |
| dhcp.pcap | 2 | Real DHCP DORA capture | DHCP Client Message Detected x2 |
| wireshark_capture.bin | 66 | The supplied Week 2 capture | TCP SYN Connection Attempt x66 |

66 SYN alerts on the supplied Week 2 capture - the same 66 connection attempts Wireshark counted

## Snort Alerts Against Real Captures

REAL OUTPUT

Verbatim alert output: [generator:SID:revision] message {protocol} source -> destination




Captured on Windows host EDITH (192.168.0.205) - 14 October 2026

## 5. The key result - 66 alerts on the supplied capture

```
$ snort.exe -A console -q -l log -c local.rules -r wireshark_capture.bin
09/21-02:51:42.436887  [**] [1:1000005:1] TCP SYN Connection Attempt [**] [Priority: 0] {TCP} 192.168.
100.128:60368 -> 192.168.100.6:135
09/21-02:51:42.443470  [**] [1:1000005:1] TCP SYN Connection Attempt [**] [Priority: 0] {TCP} 192.168.
100.128:57936 -> 192.168.100.6:49672
09/21-02:51:42.451889  [**] [1:1000005:1] TCP SYN Connection Attempt [**] [Priority: 0] {TCP} 192.168.
100.128:60372 -> 192.168.100.6:135
09/21-02:51:42.460153  [**] [1:1000005:1] TCP SYN Connection Attempt [**] [Priority: 0] {TCP} 192.168.
100.128:57940 -> 192.168.100.6:49672
09/21-02:51:42.471379  [**] [1:1000005:1] TCP SYN Connection Attempt [**] [Priority: 0] {TCP} 192.168.
100.128:60376 -> 192.168.100.6:135
09/21-02:51:42.480008  [**] [1:1000005:1] TCP SYN Connection Attempt [**] [Priority: 0] {TCP} 192.168.
100.128:57944 -> 192.168.100.6:49672
... 60 more identical alerts ...
09/21-02:52:02.167207  [**] [1:1000005:1] TCP SYN Connection Attempt [**] [Priority: 0] {TCP} 192.168.
100.128:47012 -> 192.168.100.6:445
```

Every one of the 66 alerts is the same source (`192.168.100.128`) opening a connection to the same server (`192.168.100.6`) across the 36.88-second life of the capture.

**Snort 2.9.20 Console Alerts - the supplied Week 2 capture** REAL OUTPUT

Verbatim alert output from snort.exe -A console -q -c local.rules -r wireshark_capture.bin

| Alert text as printed by Snort |
|---|
| 09/21-02:51:42.436887 [**] [1:1000005:1] TCP SYN Connection Attempt [**] [Priority: 0] {TCP} 192.168.100.128:60368 -> 192.168.100.6:135 |
| 09/21-02:51:42.443470 [**] [1:1000005:1] TCP SYN Connection Attempt [**] [Priority: 0] {TCP} 192.168.100.128:57936 -> 192.168.100.6:49672 |
| 09/21-02:51:42.451889 [**] [1:1000005:1] TCP SYN Connection Attempt [**] [Priority: 0] {TCP} 192.168.100.128:60372 -> 192.168.100.6:135 |
| 09/21-02:51:42.460153 [**] [1:1000005:1] TCP SYN Connection Attempt [**] [Priority: 0] {TCP} 192.168.100.128:57940 -> 192.168.100.6:49672 |
| 09/21-02:51:42.471379 [**] [1:1000005:1] TCP SYN Connection Attempt [**] [Priority: 0] {TCP} 192.168.100.128:60376 -> 192.168.100.6:135 |
| 09/21-02:51:42.480008 [**] [1:1000005:1] TCP SYN Connection Attempt [**] [Priority: 0] {TCP} 192.168.100.128:57944 -> 192.168.100.6:49672 |
| ... and 60 more identical alerts |

66 identical SYN alerts - one per TCP connection attempt, matching the 66 SYN-without-ACK frames Wireshark counted

This number is corroborated. Wireshark reports exactly 66 frames matching `tcp.flags.syn==1 && tcp.flags.ack==0` in the same file, and the independent Python dissector in this project reports 66 as well. Three separate tools agree, so the 66 SYN alerts are the real connection attempts in the capture - not a artefact of the rule:

| Tool | Method | SYNs counted |
|---|---|---|
| Snort 2.9.20 | `flags:S` rule, console alerts | 66 |
| Wireshark 4.6.9 | `tshark -Y "tcp.flags.syn==1 && tcp.flags.ack==0"` | 66 |
| Python dissector | `analyse_pcap.py`, raw TCP flag bits | 66 |

It also lines up with the traffic narrative: `192.168.100.128` repeatedly opening short-lived connections to ports 135 (RPC endpoint mapper) and 49672 (the dynamic RPC port that 135 allocated). That is normal Windows RPC behaviour, and it is exactly the pattern that also appears in lateral movement - which is why the value of a rule like this is that it gives the analyst a starting point rather than a verdict.

## 6. What each rule taught

| Rule | Lesson |
|---|---|
| `alert icmp any any -> any any` | The required rule. Fires on request and reply - 2 alerts per ping |
| `... itype:8` | Adding `itype:8` cuts the noise to one alert per ping. Tuning is part of detection engineering |
| `alert udp any any -> any 53` | 6 queries matched. Direction in the header matters: the reverse rule caught the 5 responses |
| `alert udp any 53 -> any any` | Proves the reply direction needs its own rule |
| `alert tcp any any -> any any (flags:S)` | The highest-value rule here - 66 alerts, one per connection attempt |
| `alert tcp any any -> any 80 (content:"GET")` | Did not fire on `http.pcap`, because the real request was `HEAD /v4/iuident.cab`. Correct behaviour: the rule said GET |
| `alert tcp any any -> any 80 (content:"HTTP/1.1")` | Matched the same packet, because every HTTP/1.1 request line carries that string whatever the method |
| DHCP `68 -> 67` and `67 -> 68` | Only the client-side rule fired (2x), because the capture contains client traffic. A rule that never fires is not necessarily broken - check whether the traffic it describes is actually present |

## 7. Live IDS mode on the VM

Reading a capture proves a rule; watching an interface proves a sensor. On the Ubuntu VM:

```
sudo snort -T -i eth0 -c /etc/snort/snort.conf        # validate
sudo snort -A console -q -c /etc/snort/snort.conf -i eth0
```

with `ip -br link` first to confirm the interface name. In a second terminal, `ping -c 5 8.8.8.8` should immediately produce `ICMP Ping Detected` alerts. This installation's `-r` runs used the same `local.rules` content, so the rule text is verified end to end.

## 8. Troubleshooting checklist

| Symptom | Cause and fix |
|---|---|
| `snort: command not found` | Use `/usr/sbin/snort`, or the package is not installed |
| `snort.exe` exits with 127 / `0xC000007B` | Wrong-architecture WinPcap DLLs next to the binary - use the 64-bit Npcap libraries |
| "Unknown rule type" | A typo in the rule header - check action, protocol, direction and brackets |
| No alerts appear | Wrong interface (`ip -br link`), the rules file is not included by the config, or the traffic is genuinely not there |
| Alerts appear twice | The rule is bidirectional (`<>`) or has no type/method filter, so request and reply both match |
| Permission denied writing logs | Run with `sudo`, or point `-l` at a directory the user can write to |
| Snort exits immediately | Run with `-T` to surface the configuration error |

## 9. What this proves to a SOC analyst

- A local rule plus a `-T` validation is the smallest complete detection engineering loop: write, test, observe, tune, increment `rev`.
- `sid` and `msg` are what make an alert actionable. `[1:1000005:1] TCP SYN Connection Attempt` tells the analyst which rule fired and why, and that SID is what a ticket or SIEM correlation rule references.
- Corroboration matters more than any single tool. The 66 alerts from Snort, 66 SYNs from Wireshark and 66 from the Python dissector agree, so the finding can be stated with confidence. Had they disagreed, the disagreement itself would have been the finding.
- Snort is signature-based: it catches known behaviour at line rate and misses anything it has no rule for. That is precisely why behaviour-based detection and ATT&CK mapping are needed alongside it - the SMB and DCERPC traffic in the Week 2 capture produced zero Snort alerts and still needed to be explained by hand.